

3.8 Equivalence of Higher-Order Functions

Dafny is able to prove the equivalence of some higher-order functions, but not all. For example, here are two equivalent functions `andThen1` and `andThen2`:

```
function andThen1<A, B, C>(f: A -> B, g: B -> C): A -> C { a => g(f(a)) }
function andThen2<A, B, C>(ff: A -> B, gg: B -> C): A -> C { aa => gg(ff(aa)) }
```

behavior?

Listing 3.34: `andThen1` and `andThen2` Functions

Dafny is able to prove their equivalence without any help:

```
lemma equivalenceAndThen<A, B, C>(f: A -> B, g: B -> C)
  ensures andThen1(f, g) == andThen2(f, g)
{}
```

Listing 3.35: Equivalence Lemma for `andThen1` and `andThen2`

Here are two more such functions `flip1` and `flip2`:

```
function flip1<A, B, C>(f: (A, B) -> C): (B, A) -> C {
  (b, a) => f(a, b)
}

function flip2<A, B, C>(f: (A, B) -> C): (B, A) -> C {
  (b, a) => var res := f(a, b); res
}
```

behavior?

Listing 3.36: `flip1` and `flip2` Functions

Dafny is able to prove that the outputted functions give the same output for the same inputs:

```
lemma equivalenceFlip<A, B, C>(f: (A, B) -> C)
  ensures forall b: B, a: A :: flip1(f)(b, a) == flip2(f)(b, a)
{}
```

Listing 3.37: Equivalence Lemma for `flip1` and `flip2` with specified arguments

However, it is unable to prove that the outputted functions themselves are equivalent:

```
// Dafny cannot prove this lemma
lemma equivalenceFlip<A, B, C>(f: (A, B) -> C)
  ensures flip1(f) == flip2(f)
{}
```

Listing 3.38: Invalid Equivalence Lemma for `flip1` and `flip2`

Therefore, Dafny is able to prove the equivalence of some higher-order functions. However, it struggles with others, despite being able to prove that the outputted functions give the same output for the same inputs.

any ideas where the limitations truly lie
(i.e. any pattern, to what it can/cannot prove)?

Chapter 4

Plans

4.1 Project Plan

The aim of this project is to build a tool on top of the Dafny program verifier that removes as many of the limitations identified in Chapter 3 as possible. Some of these limitations are out of the scope of this project. Preventing incorrect counterexamples (as described in Section 3.2) is one such limitation, as it results from problems in Boogie and the underlying Z3 solver. Similarly, the limitations related to imperative programs (as described in Section 3.7) are also out of scope, as Dafny functions provide enough complexity for this project by themselves.

I plan to build my tool in Scala. It will be called from the command line, taking as input multiple Dafny files, containing the functions to be compared and any help the user needs to provide. The tool will also require a JSON configuration file, specifying which functions must be proven to be equivalent and any other information the tool requires. The tool will attempt to create an equivalence proof and prove it using Dafny; Dafny's output will be used as the output of the tool.

The following table summarises the current development plan, broken down into milestones. Each milestone has an associated completion deadline and a description of what that milestone involves. Each milestone is described as either Necessary or Optional. If Necessary milestones take longer than I expected, I will use the time previously dedicated to Optional milestones to complete the Necessary ones.

If I have time remaining once all the project and evaluation milestones are completed, I plan to further extend our tool to handle the remaining limitations relating to Dafny functions identified in Chapter 3. This includes removing the necessity for the user to explicitly define induction schemes to prove lemmas (Section 3.5) and implementing the ability to prove user-provided auxiliary lemmas (Section 3.4).

Milestone	Description	Deadline	Necessity
1	Building the basic structure	09/02/2026	Necessary
2	Creating a CI script for Continuous Testing and Evaluation	16/02/2026	Necessary
3	Create Parser for Dafny functions	02/03/2026	Necessary
4	Copying Variants into the Main Lemma	09/03/2026	Optional
5	Generating Helper Equivalence Lemmas as Axioms	16/03/2026	Optional
6	Calling Helper Lemmas in the Main Lemma	23/03/2026	Optional
7	Calling Helper Lemmas in Equivalence Lemmas	26/03/2026	Optional
8	Proving Equivalence with one of a number of solutions	02/04/2026	Optional
9	Allowing the user to provide Auxiliary Lemmas as Axioms	06/04/2026	Optional
10	Proving equivalence of higher order functions	01/05/2026	Optional
11	Writing the Final Report	01/06/2026	Necessary

Table 4.1: Summary of the current development plan

Building the basic structure This milestone involves creating the skeleton of the tool. It should be able to take in Dafny files and a JSON configuration file. The Dafny files will contain the two functions being compared and an equivalence lemma without any user assistance. The tool will simply combine these files together, call the Dafny verifier on the resulting file, and return its output. Equivalence

lemmas with no user assistance are used here to provide an evaluation baseline for the evaluation of the remaining milestones.

Creating a CI script for Continuous Testing and Evaluation At each milestone, evaluation data must be generated and collected (see Section 4.2 for more details on this). This milestone involves creating a script to collect this data, and setting up a GitLab CI pipeline to automatically run this script on every merge. Once this is done, evaluation data should be collected for the previous milestone. Then, the equivalence lemmas should be replaced with lemmas that include all the user assistance that Dafny requires, followed by a recollection of the evaluation data, to serve as another baseline for the remaining milestones. ✓

Create Parser for Dafny functions Several of the future milestones will require an in depth understanding of the structure of the Dafny functions being compared and of the equivalence lemma provided by the user. Therefore, in this milestone, I will create parsers specifically for Dafny functions and lemmas. ✓

Copying Variants into the Main Lemma As described in Section 3.6, when one or both of the functions being compared require the user to explicitly provide variants, this must be copied into the main equivalence lemma. This milestone involves implementing this functionality. ✓

core | **Generating Helper Equivalence Lemmas as Axioms** This milestone will involve implementing the method of finding and generating equivalence lemmas for helper functions described by Milovančević and Kunčak [11] and summarised in Section 3.3. For now, these lemmas will be generated as axioms; they will be proven properly in a later milestone. ✓

core | **Calling Helper Lemmas in the Main Lemma** As described in Sections 3.3 and 3.6, helper equivalence and termination lemmas must be explicitly called in the main equivalence lemma, under the correct conditions and with the correct arguments. This milestone involves implementing this specifically for the main equivalence lemma. ✓

core | **Calling Helper Lemmas in Equivalence Lemmas** This milestone involves extending the functionality implemented in the previous milestone to also prove the generated helper equivalence lemmas. ✓

Proving Equivalence with one of a number of solutions In certain cases, including those described by Milovančević and Kunčak [11], the user may wish to prove the equivalence of a function with any one of a number of possible other functions. This milestone involves implementing this. ✓

Allowing the user to provide Auxiliary Lemmas as Axioms In certain cases, the user will need to provide auxiliary lemmas to help Dafny prove equivalence. These lemmas are specific to each individual scenario, and so automatic generation is out of scope for this project. Therefore, a reasonable alternative is to allow the user to provide auxiliary lemmas, which can then be called in the main equivalence lemma. These auxiliary lemmas will be added as axioms, as it would be difficult to prove them automatically. The burden of ensuring these lemmas are correct will be on the user. This milestone will involve implementing this. *are these not just added to the Dafny code input - seems trivial to support...*

Proving equivalence of higher order functions When proving the equivalence of functions that output lambda functions, these functions are equivalent if the lambda functions output the same values for the same inputs. This can be proven through a universal quantification over the domain of the lambda functions. This milestone involves implementing this functionality. ✓

Writing the Final Report Throughout development, I will add to the Final Report as I go along; this milestone involves completing the technical chapters in it. ✓

4.2 Evaluation Plan

In a similar manner to Section 4.1, here is a table summarising the current evaluation plan, broken down into milestones.

Milestone	Description	Deadline	Necessity
1	Creating Initial Evaluation Testsuite	Completed	Necessary
2	Setup of Evaluation Data Gathering	16/02/2026	Necessary
3	Testsuite Development	08/05/2026	Optional
4	Benchmark Comparison	11/05/2026	Necessary
5	Writing the Final Report	05/06/2026	Necessary

Table 4.2: Summary of the current evaluation plan

this is not part of eval plan

Creating Initial Evaluation Testsuite Before evaluation can begin, we need a testsuite containing pairs of functions which our tool will attempt to prove equivalent. As described in Chapter 3, I have already created an initial testsuite using the equivalence tests from the repository for the Stainless Scala verification tool[14]. This testsuite will be extended in a later milestone. *nice to have a precise testset*

Setup of Evaluation Data Gathering As described in Section 4.1, once each milestone is completed, a CI script will be used to automatically collect evaluation data. The first two milestones will provide baseline data on Dafny without any user assistance, and on Dafny with all the assistance it requires respectively. Data collected from all subsequent milestones will be compared to this baseline data. The evaluation data will consist of two categories. First, we must gather data to represent how difficult each example is for Dafny to verify. This category will include the metrics of the number of lines of code in each function being compared, the number of helper functions called and the time taken for Dafny to verify the example. Second, we must gather data to represent how much user assistance Dafny requires. This will include the metrics of the number of lines of code in the main equivalence lemma and the numbers of helper equivalence and auxiliary lemmas the user needs to provide. The aim of this project is for the metrics in the second category to decrease as much as possible. The first category metrics will ensure that we are interpreting these decreases in the correct context. These metrics are too shallow to provide a complete picture of the complexity of the examples and the level of user assistance required, however, they will provide a reasonable approximation.

Testsuite Development A second set of program equivalence tests have been gathered by Badihi, Li, and Rubin [1]. This milestone involves converting these tests from C and Java into Dafny, and appending them into my evaluation testsuite. *good idea*

Benchmark Comparison In this milestone, I will compare the results of our tool on our evaluation testsuite with the results from the original Dafny verifier, to demonstrate the improvements made. I will also compare our tool with Stainless on the subset of our testsuite that originated from their equivalence tests, to demonstrate how our tool compares with other state-of-the-art program verifiers.

Writing the Final Report Throughout development, I will add to the Final Report as I go along; this milestone involves completing the evaluation chapters in it.

not eval

how measure "level of user assistance"

different kind of user annotations have different difficulty levels to the user

+ some system profiling

Chapter 5

Ethical, Legal, Professional and Societal Issues

This project raises very few ethical, legal, professional and societal issues. The only potential issue is that our tool must obey the licenses of the any third-party libraries or tools it uses. In particular, our tool must obey Dafny's MIT license, since Dafny will be used to verify the final equivalence lemmas. It must also obey Stainless's Apache 2.0 license[14] and EqBench's MIT license[1], since these repositories ✓ will be used to form our evaluation testsuite.

what about environmental impact?

how sustainable is your tool - what are the power consumption overheads?

soundness? ← responsibility to not
say program is correct if
they can not
